RMIT UNIVERSITY

Report – DC motor unit testing

Dinh-Son Vu

Table of contents

1.Introduction

This document introduces how to drive a DC motor to verify that your hardware is working correctly. The essential components to control the DC motor are given in Figure 1:

- A microcontroller – ESP32
- A motor driver – IBT2
- A DC motor, equipped with encoders – DC12V333RPM
- A computer to code and debug the system
- A power supply – preferably a 12V-10A power supply. Each motor may use up to 3A at startup.

This document gives essential information, but it is up to the student to look for additional knowledge to build a more solid foundation.

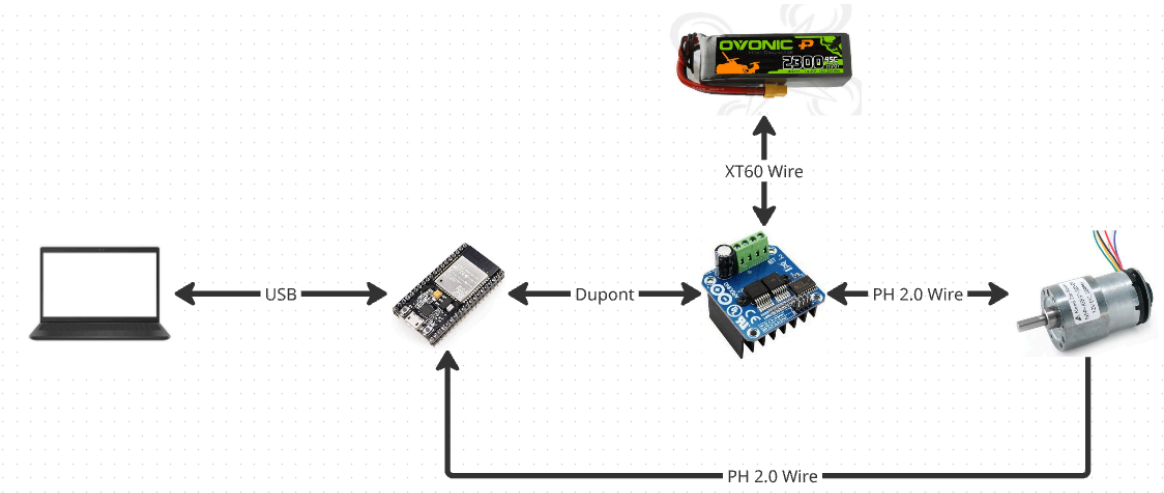


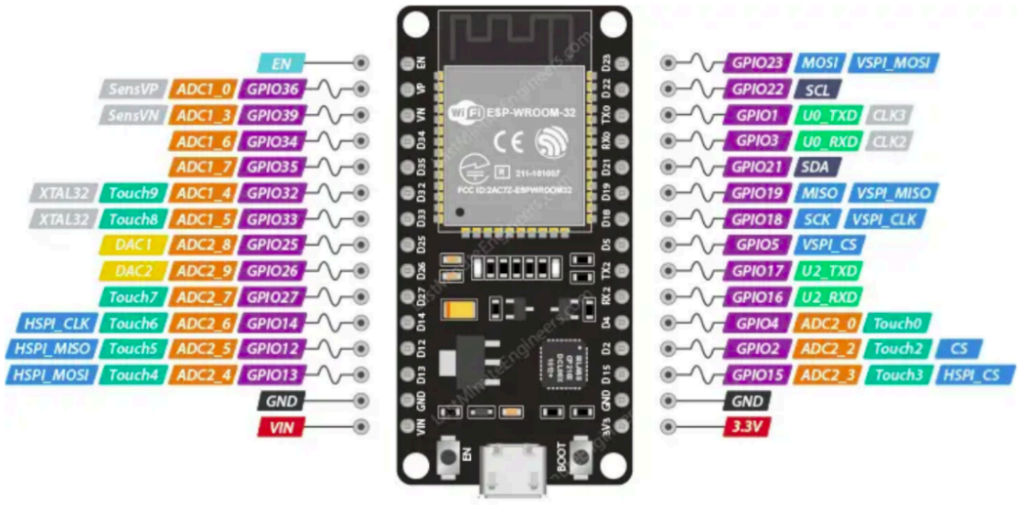
Figure 1: Essential components for driving a DC motor

2. Microcontroller

ESP32 is a microcontroller with Wi-Fi and Bluetooth. It is suitable for the IOT system. In our first example, pin 4, 16, 22, 23 will be used. One can notice that these pins are fine to use. Particular care should be considered:

- ADC1 can be used with Wi-Fi, but ADC2 cannot be used with Wi-Fi.
- Some pins cannot be used. Please refer to the following figure.

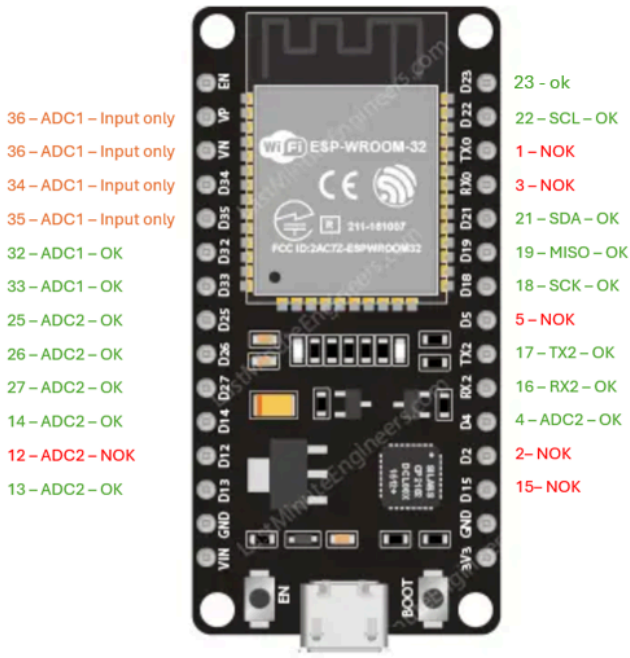
Other microcontrollers are available: Arduino, STM32, Teensy 4.1. Students are invited to compare existing systems.



Label	GPIO	Safe to use?	Reason
D0	0	!	must be HIGH during boot and LOW for programming
TX0	1	✗	Tx pin, used for flashing and debugging
D2	2	!	must be LOW during boot and also connected to the on-board LED
RX0	3	✗	Rx pin, used for flashing and debugging
D4	4	✓	
D5	5	!	must be HIGH during boot
D6	6	✗	Connected to Flash memory
D7	7	✗	Connected to Flash memory
D8	8	✗	Connected to Flash memory
D9	9	✗	Connected to Flash memory
D10	10	✗	Connected to Flash memory
D11	11	✗	Connected to Flash memory
D12	12	!	must be LOW during boot
D13	13	✓	
D14	14	✓	
D15	15	!	must be HIGH during boot, prevents startup log if pulled LOW

D15	15	!	must be HIGH during boot, prevents startup log if pulled LOW
RX2	16	✓	
TX2	17	✓	
D18	18	✓	
D19	19	✓	
D21	21	✓	
D22	22	✓	
D23	23	✓	
D25	25	✓	
D26	26	✓	
D27	27	✓	
D32	32	✓	
D33	33	✓	
D34	34	!	Input only GPIO, cannot be configured as output
D35	35	!	Input only GPIO, cannot be configured as output
VP	36	!	Input only GPIO, cannot be configured as output
VN	39	!	Input only GPIO, cannot be configured as output

Figure 2: ESP32 pinout. Note that some pins should not be used.



3. Motor Driver H-Bridge IBT2 (BTS7960)

An H-bridge is an electronic component to drive a DC motor. A power supply could be plugged directly to the DC motor, it will rotate at maximum speed, but can not be modified. Thus, the use of an H-bridge is essential. Refer to the following tutorial to use the H-bridge with a microcontroller: https://dronebotworkshop.com/dc-motor-drivers/

In our case, the H-bridge RPWM and LPWM pins are connected to the ESP32 pins 4 and 16. R_EN, L_EN, and 5V should be connected to the 3V3 of the esp32 and both grounds should be connected together.

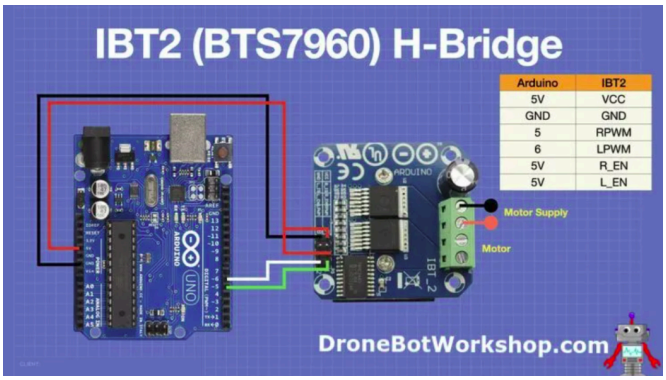


Figure 3: Basic wiring for IBT2 motor driver.

Different H-bridge modules exist: ESC, DRV8833, LN298N. However, these electronic systems may not be suited for the application with the proposed DC motor. Students are expected to justify this statement.

4. DC motor with encoder

The provided DC motor has the following specifications:

- DC motor nominal voltage: 12V
- Gearbox: 1:30
- Encoder resolution: 11 pulses per revolution

For one rotation of the output shaft, the encoder counts 330 pulses. Building a mechatronics system requires motors and sensors. The students are invited to compare different existing technologies: BLDC, stepper, AC motors, etc.

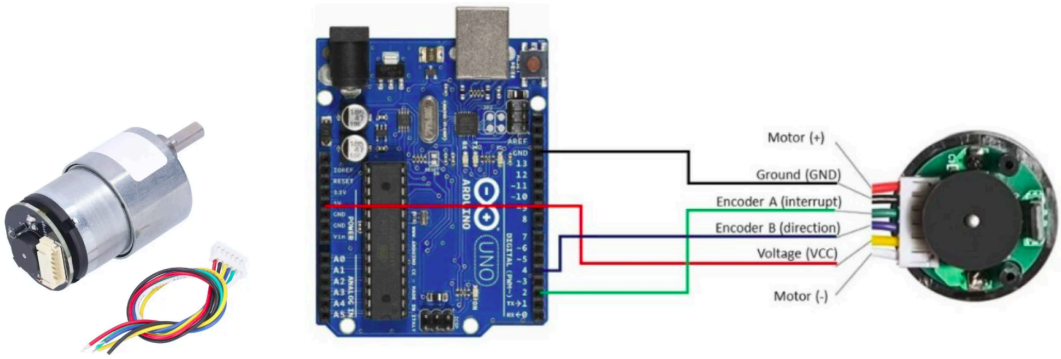


Figure 4: Basic wiring for DC motor with encoders.

A quadrature encoder gives the angular position of the motor. The functioning of the quadrature encoder should be explained by the student. In our example, the encoder A and B are connected to the esp32 pins 22 and 23.

5. Computer

The computer (PC) is connected to the ESP32 via USB. The communication between them is based on serial communication. In terms of coding environment, Arduino IDE can be used, but can be quite limited with additional libraries (encoder, PID, maybe later IMU libraries, etc.). A preferred coding interface is **VSCode**, which is used in software engineering for web development and embedded systems.
VSCode website: https://code.visualstudio.com/
However, VSCode is quite a general IDE for coding in html, css, js, cpp, etc. The preferred support for embedded systems is the add-in **platformIO**. One advantage of this tool is the ability to integrate different libraries developed by the community.
PlatformIO website: https://platformio.org/
The best would be to follow a tutorial to install VSCode, add platformIO, and get started with simple code: https://youtu.be/JmvMvIphMnY?si=IcGgeIdb5zOA86KR

6. Power Supply

In terms of power supply, the DC motor is rated with a voltage of 12V, and a current of 1A. However, at startup, the motor can use up to 3A to overcome the static friction. A fixed power supply, e.g. a power adapter, can be used at first to test the system. Later on a battery would be preferred for mobile robots. In this case, several options are available: LIPO, Li-Ion, LifePO4. It is up to the student to do their diligent literature to find the most suitable option.

7. Complete setup

Please follow the previous steps before following this last part about the complete setup.

7.1 Wiring

You may use a breadboard for unit testing purposes. However, the final robot consists of four motors and the use of a breadboard is highly undesirable. The use of a screw terminal could be a better option, but a custom PCB would be the most appropriate solution.

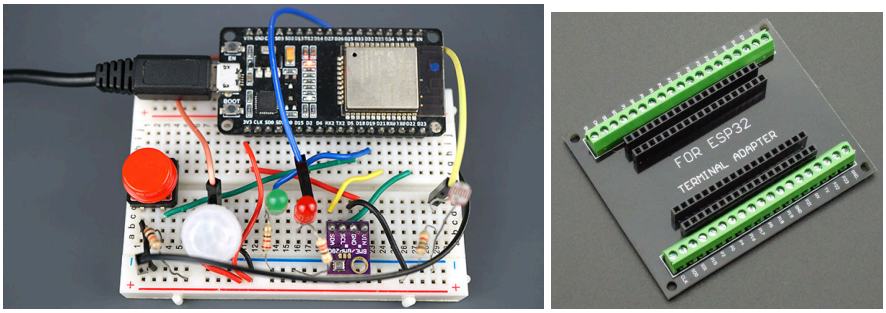


Figure 5: Breadboard will likely have loose connection. Prefer a terminal screw for ESP32

The pins used on the esp32 for the encoders are 22, 23. The pins used on the esp32 for the motor driver are 4, 16. You can refer to the file MySetup.h in the include folder while the code will be downloaded from GitHub.

7.2 GitHub code cloning

The code is available on GitHub. You may download the zip file containing the file, but a more appropriate approach is to learn how to clone the code using a terminal on your computer. This would be helpful especially while working with a terminal through ssh. The code is available at: https://github.com/ROS2-AutoBot/04-DCMotorControl

In the terminal, use “cd” to change directory and “ls” to list the items in the current folder. Then use the next function to create a folder that contains the code:
git clone https://github.com/ROS2-AutoBot/04-DCMotorControl

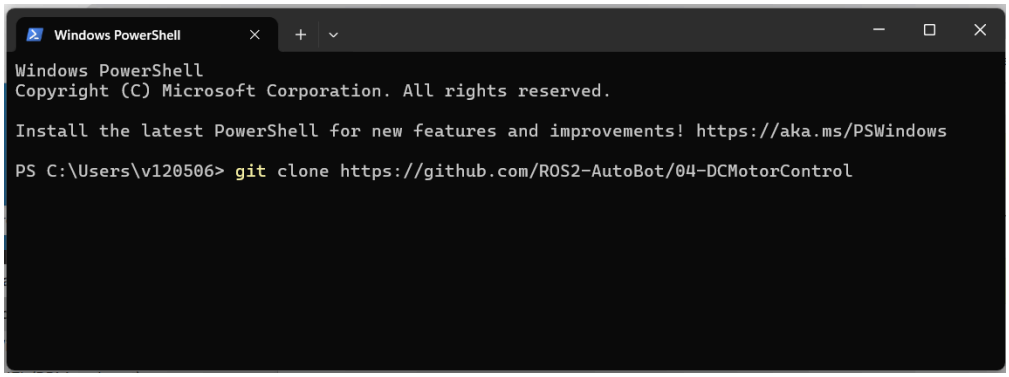


Figure 6: A PC terminal uses the command line to execute a function.

7.3 Code flashing

Flashing the code to the ESP32 is straightforward with VSCode and PlatformIO. Make sure the pin for the PWM pin for the motor driver and the encoder pin from the MySetup.h file match the wiring. With PlatformIO, select the folder that has been created “04-DCMotorControl”. Make sure that this folder contains the file platformio.ini. Then, upload the code to the esp32.

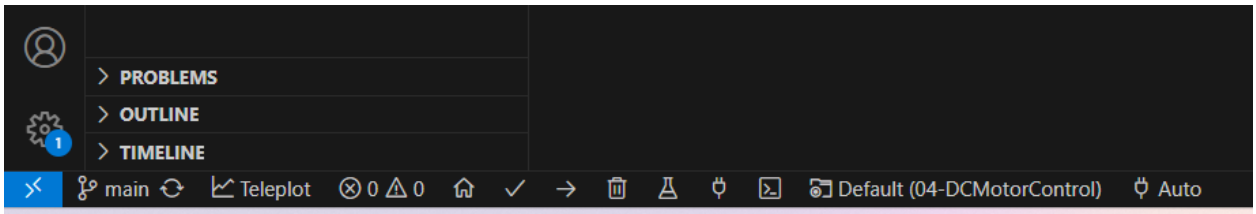


Figure 6: The interface with PlatformIO may be overwhelming. At the bottom of the screen, you may see the basic function required for compiling and flashing your code.

7.4 Testing

The file MySerial.cpp contains the code to communicate via serial with the esp32. The serial communication allows the next tasks:

- Esp32 is sending the information to the computer: speed measurement, speed reference, motor command, and encoder tick.
- The computer is sending the desired speed reference to the esp32.

Open the serial communication and type “q10”: normally you should see the DC motor rotating at 10 rad/s. You may try other values as well. An important part of the unit testing is to verify that the actual speed of the motor is correct: you may ask for a reference speed of 10 rad/s. The encoder shows that the speed is 10 rad/s, but the motor may rotate at a totally incorrect speed. The use of a tachometer as an external source of measurement would be preferred, but you may set a relatively reasonable speed to manually calculate the speed of the motor. For instance, you may set a speed of 6.28 rad/s (about 1 rotation per second) and count the actual rotation of the motor shaft per second. This would give a first approximation about the correctness of the motor speed.

An interesting axis of analysis would be control systems. You may want to tune the PID controller depending on your desired dynamics. It is possible to visualize the data with the tool called “teleplot” in VSCode. With the data collection regarding the motor speed and the motor command, it would be possible to:

- Perform model identification
- Use Simulink to select the PID gains to respect overshoot, rise time, settling time, and steady-state error specification.
- Use these found PID gains with the real system and compare the behaviour between the modelled system and the real physical one.